**Title:** KillSwitch Gateway: Finding, Containing and Undoing One Agent's Damage in a Network of Agents

**Team Lead:** Sharanya Nair

**Abstract:**

When one agent in a network starts corrupting records, spending money it should not, or obeying an instruction hidden in a document it read, nobody can say what changed, which agent changed it, or how to undo it without throwing away what the other agents got right. This project builds a gateway that sits in front of an agent network and answers all three. Every action carries the identity of the agent that took it, a monitor watches for behaviour that does not fit, one agent can be quarantined while the rest keep working, and a rollback reverses that agent's damage and nothing else. It answers one sub-theme: The Kill Switch.

**Introduction:**

Stopping an agent is not difficult. Switching off the whole network is a single command. What is difficult is stopping only the one that went wrong, and then working out which of the last few hundred changes in a shared database were its fault and which were somebody else's good work. The stopping is the overhead. The untangling is the work.

A safety layer can do both, but only if it sees every action as it happens, and only if it sits outside the agents so that it can protect a system somebody else wrote. That is the gap this project closes.

**Problem Statement:**

The Kill Switch asks for the layer that answers three questions when an agent goes wrong. Three things make that harder than it sounds.

1. **What changed is not recorded anywhere that can be trusted:** Agents log their own activity, which is exactly what cannot be believed when one of them is the problem. The record has to be made outside the agent, at the point where actions happen, and show which fields changed and what they held before. It also has to be tamper-evident, so history cannot be quietly edited afterwards.
2. **Which agent changed it is not the same as who is connected:** A shared credential says the network acted, not which member of it. Every action has to carry the identity of the agent that took it, and be checked against what that agent is supposed to be doing at all.
3. **Undoing is not the same as restoring a backup:** Restoring a snapshot erases every healthy agent's work since the damage began, including later edits to the very same record. Rolling back one agent means working out what each affected field would hold had that agent never acted, and writing exactly that.

**Proposed Solution:**

Four pieces.

1. **Gateway and Identity:** A gateway sits in front of the network and every tool call passes through it carrying the key of the agent that made it. Each agent has a declared scope, and a call outside it is blocked and recorded. Because the gateway is in front of the agents and not inside them, it can protect an agent system it did not build.
2. **Ledger:** Every call, whether allowed, blocked or rejected, is written to an append-only ledger with the before and after value of each field it touched. Each entry is hashed together with the one before it, so changing or deleting history breaks the chain and is detected.
3. **Monitor and Quarantine:** A monitor watches behaviour rather than known faults: bursts of spending, many records changed at once, abnormal write rates, repeated invalid calls, and instruction-like text in a document an agent has read followed by a risky action. When one agent trips it, that agent alone is cut off at the gateway. Its later attempts are logged as evidence and the other agents keep working.
4. **Selective Rollback and Audit:** The operator sees a preview of what would be reverted and how many healthy actions would be kept. Rollback rebuilds each affected field from its original value by replaying only the surviving actions, so a healthy agent's later change to the same field stays. An independent audit then recomputes every touched field from the ledger and compares it with the live data. Stretch goal: a learned per-agent baseline that catches slow faults staying under fixed thresholds.

**Implementation Plan:**

Three phases. A working prototype of the core already exists and is the starting point for each.

1. **Phase 1: Gateway, Identity and Ledger**
    - Route every tool call through the gateway with per-agent keys and scopes, and record each one with field-level before and after values in a hash-chained ledger.
    - Put an MCP front-end on the gateway so it can sit in front of an agent network we did not write.
2. **Phase 2: Monitor and Quarantine**
    - Implement behavioural rules for spend, mass modification, write rate, invalid calls and prompt-injection taint, with single-agent quarantine and a time-to-containment figure counted from when the fault started, not from when it was noticed.
    - Add the learned per-agent baseline and test it against faults we did not write ourselves.
3. **Phase 3: Rollback, Audit and Red Team**
    - Make rollback selective, with a preview and an independent audit of the result.

    - Run a red team of three faults against a timed run, then check two things: was every change put right, and did every healthy agent’s work survive.

**Expected Outcomes:**

What we expect to show:

- An agent network we did not write, protected by the gateway, with one agent quarantined and the others still working.
- Each fault contained in seconds, measured from when it started rather than when it was noticed. In the prototype, three injected faults were each contained in under two seconds.
- Rollback that leaves every healthy agent’s work intact, confirmed by an audit that recomputes the state from the ledger and finds no drift. The prototype’s audit found none after either rollback it needed.
- Every action, including the faulty ones, traceable to a named agent in a ledger that shows when it has been tampered with.

**Conclusion:**

The interesting part of this is not that a misbehaving agent can be switched off. It is that one agent can be switched off, and its damage undone, without losing anything the rest of the network did correctly. That comes down to whether every change can be attributed to the agent that made it, and reversed without disturbing the others. Detection is the part that demonstrates well. Rolling one agent back after a neighbour has since edited the same record, and having the data still be right afterwards, is the part that decides whether anybody would trust it. That is where we expect to spend our time.

**Keywords:** Agent Safety, Multi-Agent Systems, Audit Ledger, Quarantine, Selective Rollback, Prompt Injection, The Kill Switch, SEED Hackathon 2026.